[12]

Pull Requests (Merge Requests)

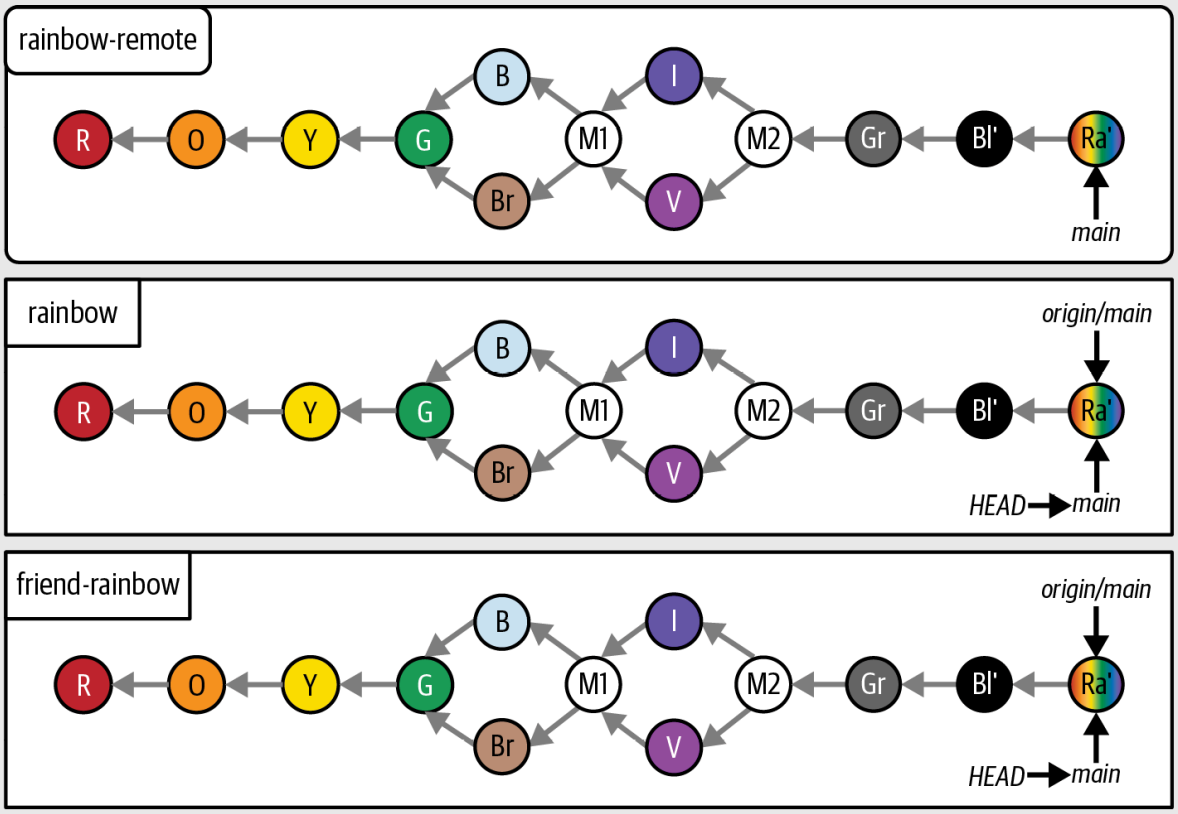
Up until now, we have only covered how to integrate changes from one branch into another when working with a local repository.
In this chapter you are going to learn about pull requests, a helpful tool for collaboration that allows you to integrate changes from one branch into another in a remote repository. Along the way, you will also learn a handy trick for how to more easily define upstream branches for new local branches.
Additional resources to assist you as you work through this chapter are available in the Learning Git repository (*https://github.com/gitlearningjourney/learning-git*).

State of the Local and Remote Repositories

At the start of this chapter, you should have two local repositories called rainbow and friend-rainbow and one remote repository called rainbow-remote. All three of these repositories should be in sync, with the same commits and branches.

Visualize it 12-1 shows the state of the local and remote repositories in the Rainbow project with all the commits that were made from Chapter 1 to Chapter 11.

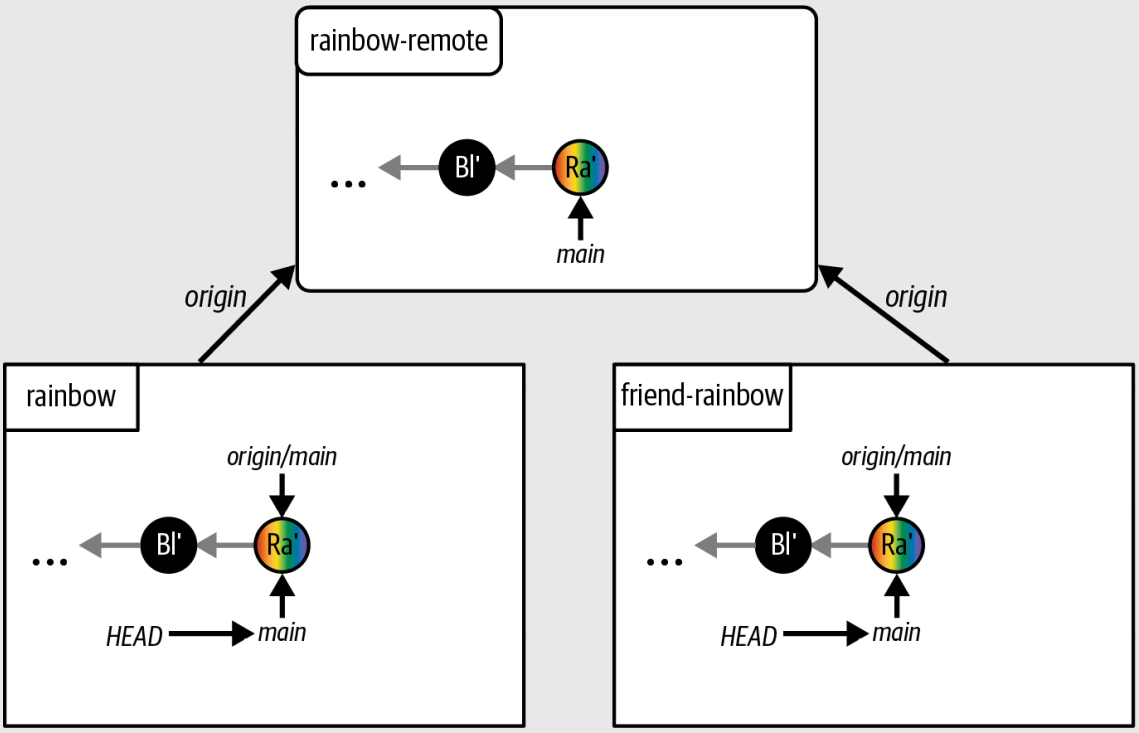
[VISUALIZE IT 12-1]



The Rainbow project at the start of Chapter 12 with all the commits since Chapter 1

To focus on the commits you are going to make in this chapter, from here on I will simplify the Visualize It diagrams and show only the last two commits that are a part of the main branch in all the repositories, which are the black commit and the rainbow commit. This representation is shown in Visualize it 12-2.

[VISUALIZE IT 12-2]



A simplified representation of the Rainbow project at the start of Chapter 12, showing just the last two commits on the main branch in all the repositories

Introducing Pull Requests

A *pull request* (also referred to as a merge request) is a feature offered by a hosting service that allows you to share work you have done on a branch with your collaborators, potentially gather feedback on that work, and finally integrate that work into the project remotely on the hosting service. Although pull requests are not a feature of Git but of the hosting services that host projects using Git, they are so useful in day-to-day work with Git that I wanted to introduce them.

Pull requests can be integrated by merging or rebasing, but the default (and most common) option is merging, so we will stick to that for the example in this chapter. Throughout the rest of the chapter I will refer to the process of integrating a pull request as "merging a pull request."

When you create a pull request, you may say that you "open" a pull request. Once the pull request has been reviewed, approved, and merged, you "close" it. You may also

close a
pull request if you decide not to merge it and you want to remove it from the list of open pull requests (for example, if the pull request does not get approved).

The pull request process can be split into nine steps:

1. Create a branch in the local repository.
2. Add work by making commits on the branch.
3. Push the branch to the remote repository.
4. Create (or open) a pull request in the hosting service.
5. Get the pull request reviewed and potentially incorporate any feedback from other people into the pull request.
6. Get the pull request approved.
7. Merge the pull request.
8. If it is a topic branch (feature branch), delete the remote branch.
9. Pull the changes to sync your local repository with the remote repository, and clean up by deleting the local branch and the remote-tracking branch.

[NOTE]

If there are merge conflicts between the branches in the pull request, you will not be able to merge it. You must first resolve the merge conflicts. Some hosting services provide support for resolving merge conflicts on their website; however, it is most common to resolve merge conflicts in your local repository, using the process outlined in Chapter 10.

Before we get into an example, let's briefly touch upon some things you'll need to keep in mind when working with a hosting service.

Hosting Service Specifics

The specific steps for creating and managing pull requests with GitHub, GitLab, and Bitbucket are different, and you may need to consult your hosting service's documentation to complete some of the Follow Along exercises in this chapter. For additional resources, go to the Learning Git repository (*https://github.com/gitlearningjourney/learning-git*).

Terminology may also differ between hosting services: notably, while GitHub and Bitbucket use the term "pull request," GitLab refers to the same feature as a "merge request." Keep these differences in mind when carrying out the Follow Along exercises in this chapter.

> **[NOTE]**
>
> Even though "pull" is in the name, pull requests are not actually related to the git pull command.

Finally, in Chapter 6, when we walked through choosing a hosting service and setting up HTTPS or SSH access, I recommended that you use a personal hosting service account rather than a company account. This is because it is possible to configure additional settings in a hosting service for the pull request creation and approval process. For example, you (or the company you work for) may define certain requirements for creating a pull request, or you may set restrictions on who is able to approve pull requests or how many individuals in a team need to approve a pull request before it can be merged.

The examples in this book assume that no extra settings have been configured. However, keep this in mind if you're using a company account, as there may be additional requirements and restrictions in place that affect your ability to create and manage pull requests.

Next, let's discuss why you might want to use pull requests in the first place.

Why Use Pull Requests?

Pull requests facilitate communication and collaboration on Git projects by providing an easy mechanism to review work. They have a useful commenting feature that allows you and your collaborators to add comments to specific lines in the files of a project, respond to these comments, and start discussion threads. This helps organize the review process.

Since pull requests are managed entirely in the hosting service UI, they also allow non-Git users to provide feedback on Git projects—you don't need to know how to use Git to make comments on a pull request! Let's look at Example Book Project 12-1 to see an example of how using pull requests can be useful.

EXAMPLE BOOK PROJECT 12-1

Suppose I make a branch off the main branch called chapter_nine to work on chapter 9 of my book. I make two commits on that branch, commit W and commit X, then I push my local chapter_nine branch to the remote repository, creating a remote chapter_nine branch. The state of the local and remote repositories is illustrated in Figure 12-1.

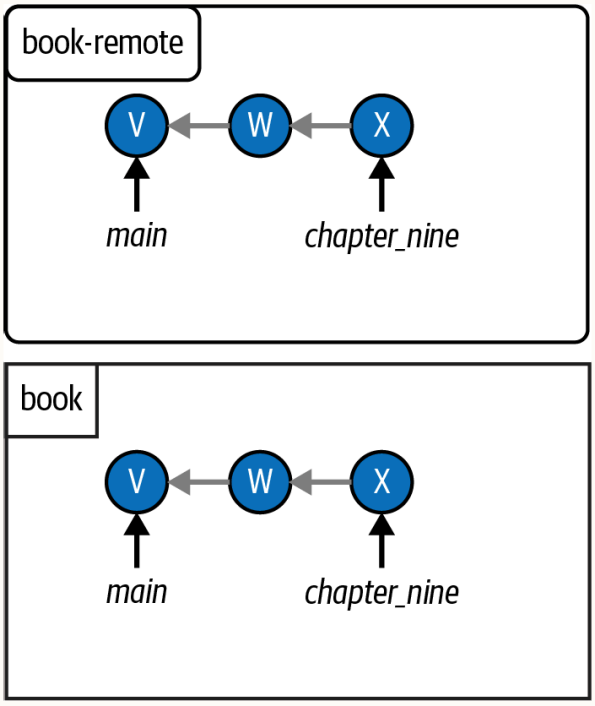


FIGURE 12-1

The Book project after I push the chapter_nine branch to the remote repository

Recall that I've agreed with my editor that they must review my work before I merge it into the main branch. This means they need to review the new chapter_nine branch. One option is for them to clone the remote repository on their local computer and check out the branch there, in order to view my updated file. However, this doesn't give them an easy way of providing me with feedback and comments. Also, let's assume that my editor hasn't yet learned how to use Git and therefore would struggle with cloning and working in a local repository.

Instead, I'll make sure my editor has access to the remote repository, then I'll make a pull request in the remote repository to merge the remote chapter_nine branch into the remote main branch. I can either send my editor the URL to the pull request or simply tell them to go to the remote repository and find the pull request titled "Chapter 9 updates."

My editor can then use the commenting feature to ask questions and provide feedback. Suppose they notice an inconsistency in the chapter that I need to fix, so they leave a

comment in the pull request and let me know that I should review their feedback. After fixing the issue in the chapter in my local repository, I make another commit on the local chapter_nine branch (commit Y) and push it to the remote repository. This will automatically update the remote chapter_nine branch, and it will therefore also automatically update the pull request. Figure 12-2 shows the updated state of the local and remote repositories.

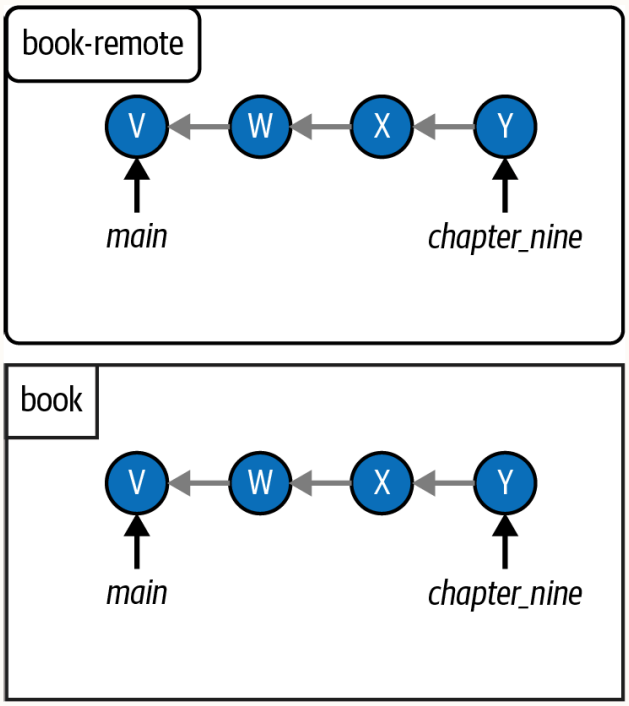


FIGURE 12-2

The Book project after I make commit Y on the chapter_nine branch and push it to the remote repository

The fact I made the pull request makes it easy for my editor to provide me with feedback on my Book project, and for me to share my updates after I incorporate their feedback.

Example Book Project 12-1 illustrates why pull requests are so useful. Next, let's take a look at how pull requests actually integrate work.

Understanding How Pull Requests Are Merged

As you know, there are two types of merges in Git: a *fast-forward merge* happens when the developmental histories of the source branch and the target branch have not diverged, whereas if they have, a *three-way merge* happens and a merge commit is made.

You walked through examples of performing both kinds of merges in Chapters 5 and 9, respectively.

By default, merging remotely is different from merging locally. The default setting for most hosting services is that a remote merge with a pull request happens with a merge option called *non-fast-forward*. With this option, even if the development histories of the source branch and the target branch have *not* diverged, a merge commit will *still* be made.

Merges made with the non-fast-forward option are sometimes referred to as *explicit merges*. This is because they always explicitly show where a merge happened with a merge commit.

It is possible to change the setting on a hosting service to merge with a different option or approach. However, since the non-fast-forward option is the most common, that's what we will cover in this book. Let's revisit the scenario from Example Book Project 12-1 in Example Book Project 12-2 to see how pull requests usually merge work.

EXAMPLE BOOK PROJECT 12-2

In Example Book Project 12-11, I described how I made a pull request titled "Chapter 9 updates" so my editor could review the work I had done on the chapter_nine branch that I wanted to merge into the main branch. Figure 12-3 shows the state of the Book project at this point.

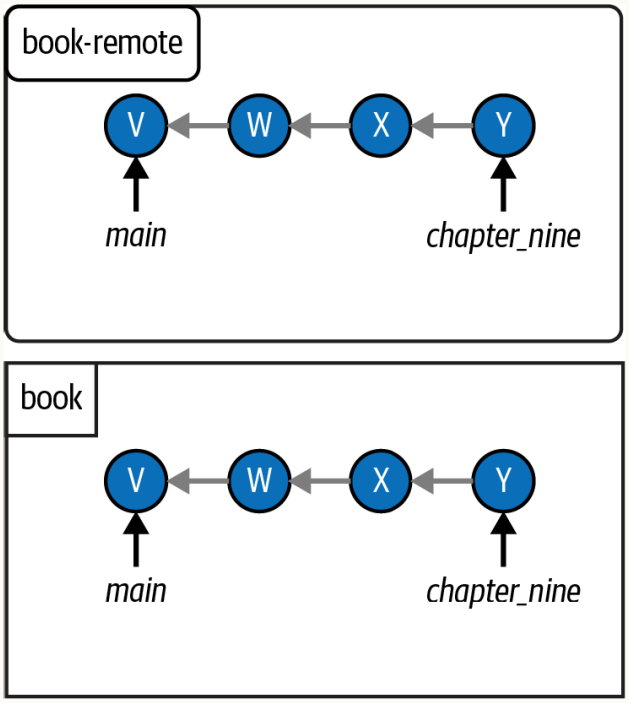


FIGURE 12-3

The state of the local and remote repositories in the Book project

After my editor reviews the latest work that I pushed to the remote chapter_nine branch, which is represented by commit Y, they no longer have any more feedback for me. So, they select the button on the hosting service's website to approve the pull request.

This means that I can merge the remote chapter_nine branch into the remote main branch by selecting the button on the website to merge the pull request. Since I have the default settings in place, this will be a non-fast-forward merge. Even though the development histories of the remote chapter_nine branch and the remote main branch have not diverged, a merge commit will still be made. This is represented as commit M in Figure 12-4.

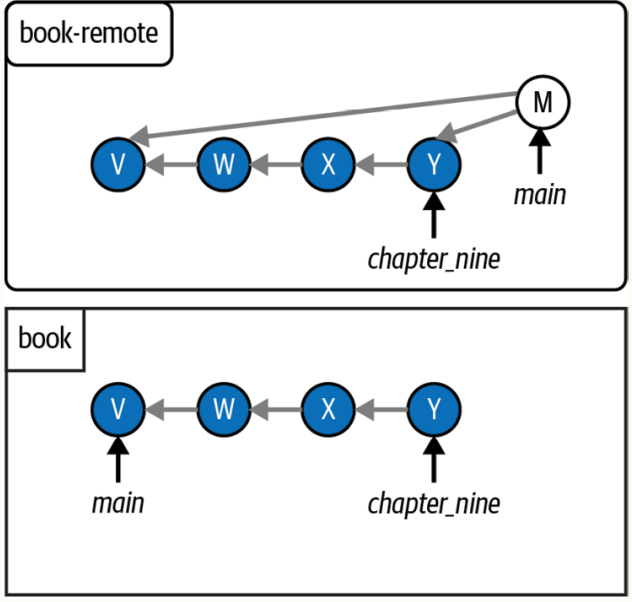


FIGURE 12-4

The Book project after I merge the pull request to merge the chapter_nine branch into the main branch

The parent commits of the merge commit M are commit V, where the main branch was pointing before the merge, and commit Y, the latest commit on the chapter_nine branch.

The final actions I need to take are to delete the chapter_nine branch and pull the most up-to-date version of the main branch from the remote repository to my local repository, so that my local main branch is in sync with the remote repository. This is illustrated in Figure 12-5.

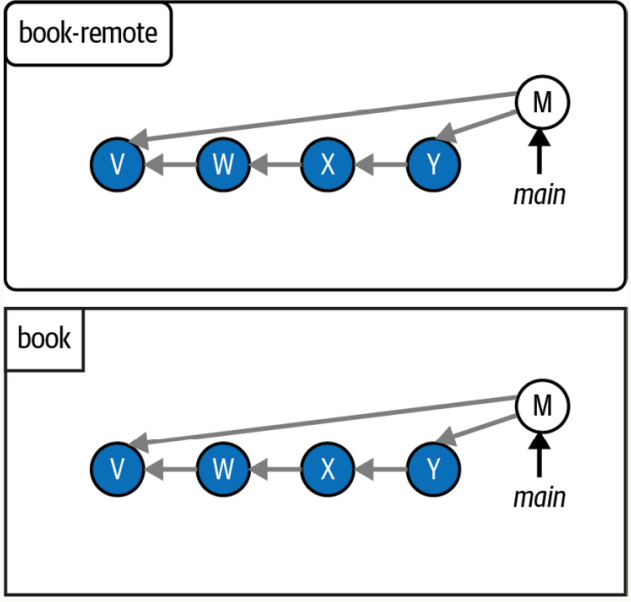


FIGURE 12-5

The Book project after I delete the chapter_nine branch and pull the changes from the remote main branch to my local main branch

---

Now that we have covered an example of a pull request in Example Book Project 12-2, let's turn to the Rainbow project so you can practice going through the pull request process.

Preparing to Make a Pull Request

To go over an example of a pull request, you will start by completing steps 1 and 2 of the pull request process (described in "Introducing Pull Requests" on page 233) in the rainbow repository.

In Follow Along 12-1, you will use the git switch command with the -c option, introduced in "Creating a Branch and Switching onto It in One Go" on page 86, to create and immediately switch onto a new branch called topic. Then you will add work to it.

[NOTE]

The reason you will use the generic name topic for the new branch is because, as mentioned in “Why Do We Use Branches?” on page 44, it is common to refer to branches that are created to work on a specific part of a project in Git as *topic branches* (or *feature branches*). In a real Git project, the branch name would usually contain a brief description of the feature or topic you’re working on.

[FOLLOW ALONG 12-1]

1. rainbow $ **git switch -c topic**
   Switched to a new branch 'topic'

2. Open the rainbow project directory in your text editor. In the othercolors.txt file, on line 4, add “Pink is not a color in the rainbow.” and save the file.

3. rainbow $ **git add othercolors.txt**

4. rainbow $ **git commit -m "pink"**
   [topic 4c35a5c] pink
   1 file changed, 1 insertion(+), 1 deletion(-)

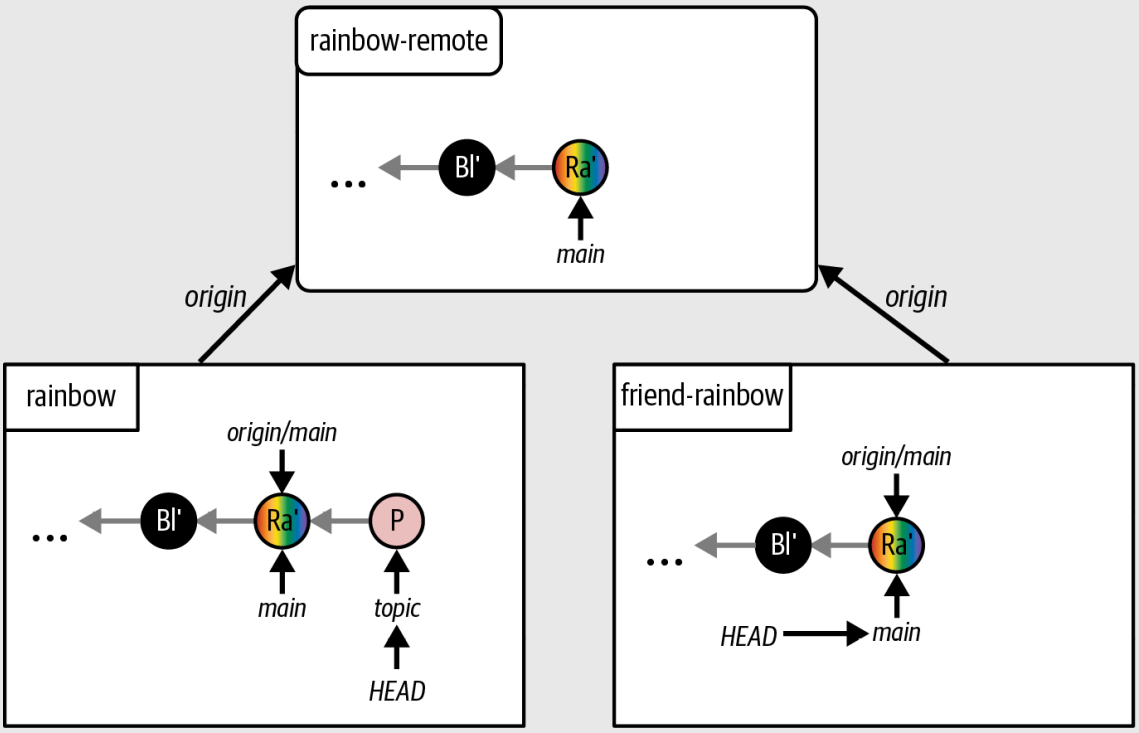
5. rainbow $ **git log**
   commit 4c35a5c02c3dc03f044cbdfdbb0ae55161af6a86 (**HEAD -> topic**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sun Jul 3 14:16:01 2022 +0200
   pink
   commit 7c09136bcbfdd9f638ed13c6653e06451579d21c (**origin/main, main**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sun Feb 20 10:10:11 2022 +0100
   rainbow

What to notice:

- You created the pink commit on the topic branch in the rainbow repository.

This is illustrated in Visualize it 12-3.

[VISUALIZE IT 12-3]



The Rainbow project after you make the pink commit on the topic branch in the rainbow repository

The topic branch is a new local branch. It does not have an upstream branch defined for it. (Recall that an *upstream branch* is the remote branch that a particular local branch tracks; you learned about this concept in Chapter 7, and you learned about defining an upstream branch in Chapter 9.)

Next, you will carry out step 3 of the pull request process, which is to push your work to the remote repository. In the process, I'll introduce an easier way to define upstream branches.

An Easier Way to Define Upstream Branches

In Chapter 9, you learned how to use the git branch -u <shortname>/<branch_name> command to define an upstream branch. But now, as we near the end of this book, I'll let you in on a little secret. It's really common for Git users to forget to define upstream branches. Or to skip setting them up out of sheer laziness.

However, there is a helpful trick that Git users often use to define upstream branches when they first push a new branch to a remote repository. If you use the git push command without any arguments on a branch that does *not* have an upstream branch defined, then Git will issue a warning in the output of the command. The warning will provide you with the command to use to set one (git push --set-upstream <shortname> <branch_name>), and it will suggest a shortname and branch name that you might want to use (Git assumes that you'll want the remote branch to have the same name as the local branch). If you copy and paste this command into the command line and execute it, then you can accomplish two tasks in one go: you push the local branch to the remote repository and define an upstream branch for it.

Go to Follow Along 12-2 to carry out a hands-on example of this.

[FOLLOW ALONG 12-2]

1. rainbow $ **git branch -vv**
   main 7c09136 [origin/main] rainbow
   * topic 4c35a5c pink

2. rainbow $ **git push**
   fatal: The current branch topic has no upstream branch.
   To push the current branch and set the remote as upstream, use
   git push --set-upstream origin topic

3. Copy the git push --set-upstream origin topic command included in the output of the git push command in step 2, and paste it in the command line.

4. rainbow $ **git push --set-upstream origin topic**
   Enumerating objects: 5, done.
   Counting objects: 100% (5/5), done.
   Delta compression using up to 4 threads
   Compressing objects: 100% (3/3), done.
   Writing objects: 100% (3/3), 314 bytes | 314.00 KiB/s, done.
   Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
   remote: Resolving deltas: 100% (1/1), completed with 1 local object.
   remote:
   remote: Create a pull request for 'topic' on GitHub by visiting:
   remote: https://github.com/gitlearningjourney/rainbow-remote/pull/new/topic
   remote:
   To github.com:gitlearningjourney/rainbow-remote.git
   * [new branch] topic -> topic
   branch 'topic' set up to track 'origin/topic'.

5. rainbow $ **git branch -vv**
   main 7c09136 [origin/main] rainbow
   * topic 4c35a5c [origin/topic] pink

6. rainbow $ **git log**
   commit 4c35a5c02c3dc03f044cbdfdbb0ae55161af6a86 (**HEAD -> topic**, **origin/topic**)

[FOLLOW ALONG 12-2]

Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Jul 3 14:16:01 2022 +0200
pink
commit 7c09136bcbfdd9f638ed13c6653e06451579d21c (**origin/main**, **main**)
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Feb 20 10:10:11 2022 +0100
rainbow

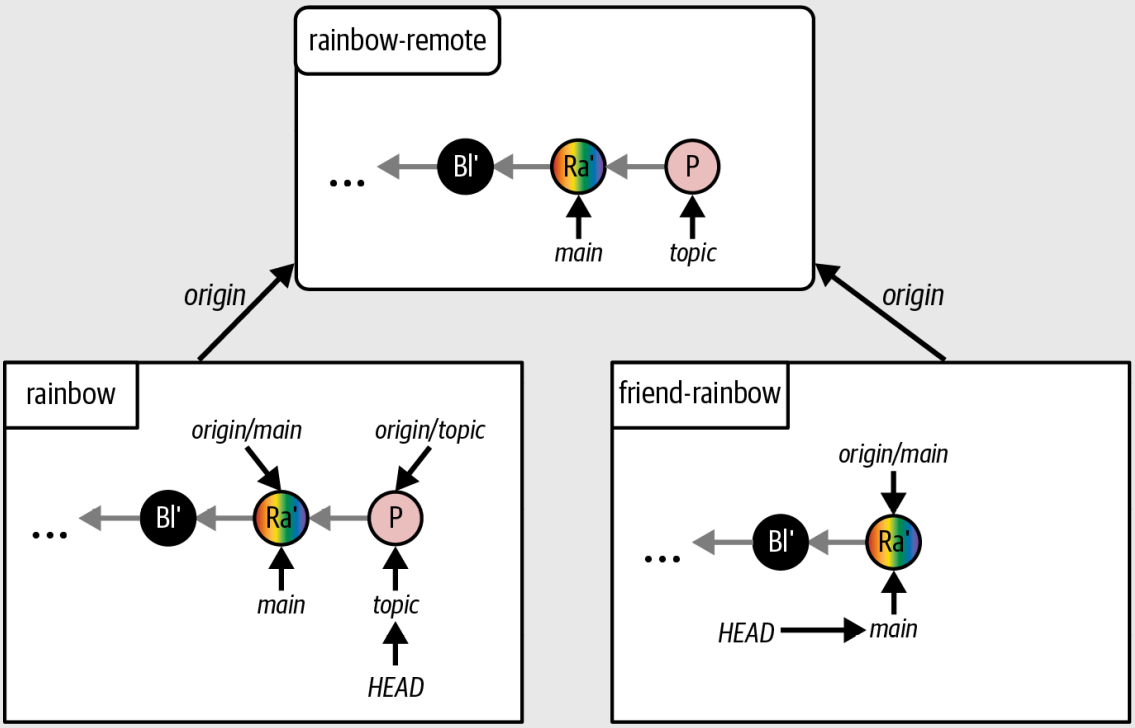
7 Go to the rainbow-remote repository on your hosting service and refresh the page. You should see the new topic branch you created and if you select to view the commits on the topic branch, you will also see the pink commit.

What to notice:

- In step 2, the output of the git push command provides a warning: The current branch topic has no upstream branch. It also provides the following instructions: To push the current branch and set the remote as upstream, use git push --set-upstream origin topic.
- In step 4, the git push output recommends that you make a pull request for the new branch: Create a pull request for 'topic' on GitHub by visiting: https://github.com/gitlearningjourney/rainbow-remote/pull/new/topic.

The current state of the repositories after Follow Along 12-2 is illustrated in Visualize it 12-4.

[VISUALIZE IT 12-4]



The Rainbow project after you push the topic branch to the rainbow-remote repository and define the upstream branch for the local topic branch in one go

You just saw how you can use the git push command to easily define an upstream branch while pushing a branch to the remote repository. Next, you will carry out step 4 of the pull request process.

[NOTE]

This trick assumes that you want the upstream branch of your local branch to be a remote branch of the same name in the remote repository. If this is not the case, you may have to edit the git push command that Git provides you to indicate the alternative remote branch that you want to set as the upstream branch.

Creating a Pull Request on a Hosting Service

When you create a pull request, you have to define the source branch and the target branch. In the Rainbow project example, topic is the source branch and main is the target branch. In other words, you are merging topic into main. To create the pull request, you will have to use your hosting service's UI. For additional resources, go to the Learning Git repository (*https://github.com/gitlearningjourney/learning-git*).

On the web page to create the pull request, you will have the opportunity to enter information about the pull request. The only required field for most hosting services is a title. As with commit messages and the names of branches, if you're working with a team on a Git project you should check with them about whether they have conventions established for pull request titles. For the Rainbow project example in this book you will use the "Adding the color pink" and leave all the other fields empty.

In GitLab and Bitbucket, when making a pull request you may also select an option that will automatically delete the branch that you're merging after the pull request is merged. It's up to you whether you want to select this option (if it is available on your hosting service). In any case, after you merge the pull request you will make sure the remote branch is deleted (if it is still there), as this is step 8 in the pull request process.

Now, go to Follow Along 12-3 to create your pull request.

[FOLLOW ALONG 12-3]

1. Go to your remote repository on your hosting service. Follow the steps to create a pull request to merge topic (the source branch) into main (the target branch). The hosting service may prefill the pull request title field with the commit message of your last commit, which in your case is "pink". You should change the title to "Adding the color pink". For additional resources, go to the Learning Git repository (*https://github.com/gitlearningjourney/learning-git*).

Now that you have created your first pull request, it's time to move on to steps 5 and 6 of the pull request process and get it reviewed and approved.

Reviewing and Approving a Pull Request

Pull requests provide an opportunity for collaborators on a project to review your work, contribute to it, or simply approve it. The hosting service UI usually provides a way to view the lines that have been changed in each file that has been modified and how they have been changed, using colors and symbols to display this information. Figure 12-6 shows how a hosting service might indicate that you added a sentence on line 4 of the othercolors.txt file about the color pink.

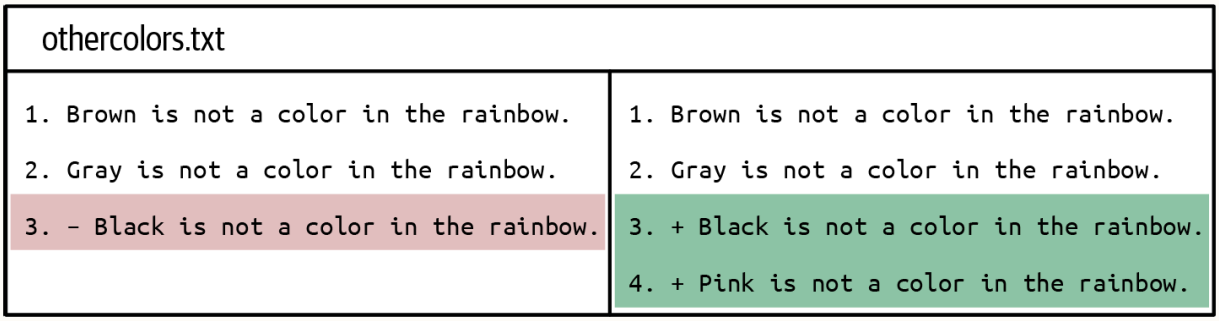


FIGURE 12-6

An example of how a hosting service displays the files changed in a pull request

When collaborators review your work, they can choose to leave comments on the pull request. If your pull request has comments, you can review them, potentially make changes, and push one or more additional commits to the branch that you are merging. This will automatically update the pull request with the new commits.

A collaborator can also decide to pull the branch to their local repository and make additional changes on the branch by adding commits and pushing them to the remote repository. Again, the pull request will automatically update with their additional commits.

In the Rainbow project example, suppose you've told your friend that you made the pull request and asked them to review it. Your friend will go to the pull request and take a look at it, decide that they're happy with the changes you made as they are, and approve the pull request.

Normally if you had a collaborator reviewing your pull request, they would log into their own account on the hosting service, then review and approve it by selecting the "approve" button in the hosting service's UI. However, since you're simulating that you are two people, this "approve" button might not appear in the UI. In this case, you will just have to pretend that your friend approves the pull request.

Go to Follow Along 12-4 now to simulate your friend reviewing and approving the pull request.

[FOLLOW ALONG 12-4]

1. Pretend you are your friend, and go to the pull request on your hosting service.
2. In the pull request, view the files that were changed. If necessary, consult the hosting service's documentation for details on how to do this.
3. "Approve" the pull request.

Once the pull request has been approved (figuratively, if not literally), you may proceed to step 7 of the pull request process.

Merging a Pull Request

Acting as yourself again, you can now go back to your hosting service and carry out the steps to merge the pull request. This usually just consists of going to the pull request in the UI and selecting the relevant button. Go to Follow Along 12-5 to merge your pull request now.

[FOLLOW ALONG 12-5]

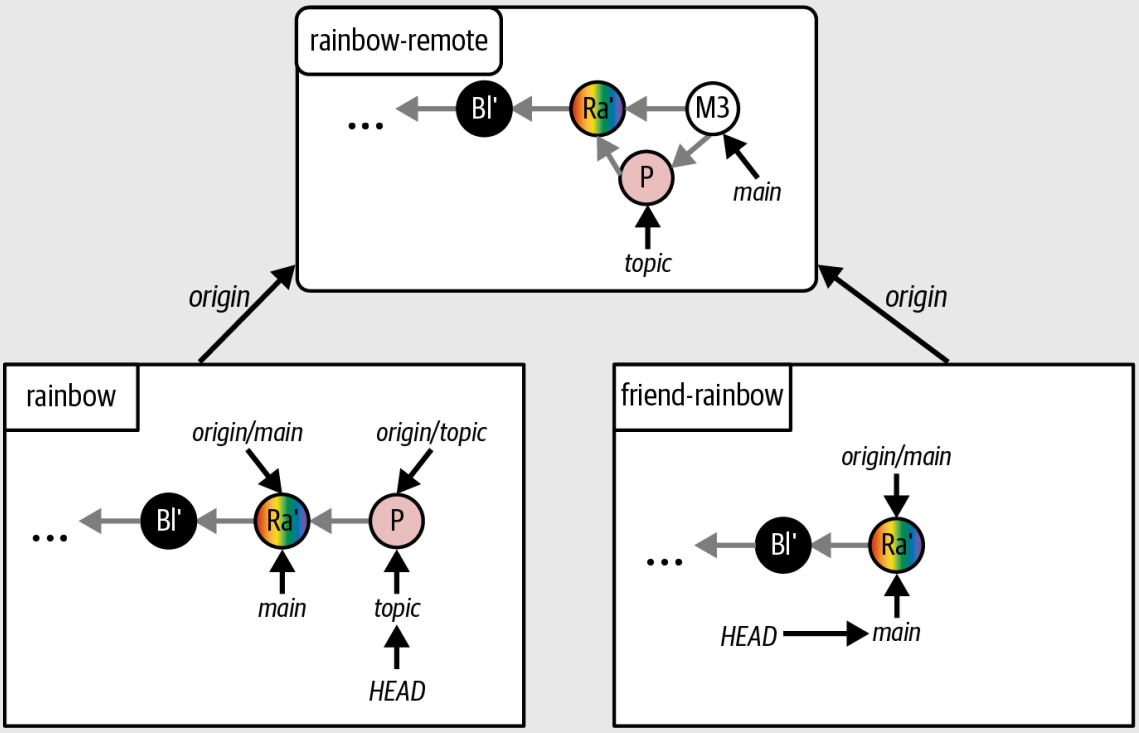
1. Follow the steps for your hosting service to merge the pull request.
2. Review the list of commits in the remote repository. Find the latest merge commit and select it. Make a note of its commit hash as well as the commit hashes of its two parent commits.

What to notice:

- You merged the remote topic branch into the remote main branch and created a merge commit.

This is represented in Visualize it 12-5; the new merge commit is labeled M3.

[VISUALIZE IT 12-5]



The Rainbow project after you merge the pull request in the remote repository

As mentioned earlier in this chapter, this merge is performed with the default non-fast-forward option. This means that even though the development histories of the remote topic branch and the remote main branch had not diverged, the merge still created a merge commit.

In Visualize it 12-5, you can see that the remote topic branch is still in the rainbow-remote repository and the local topic branch and origin/topic remote-tracking branch are still in the rainbow repository. Moving on to the eighth step of the pull request process, let's go ahead and delete the remote topic branch.

Deleting Remote Branches

If the branch you have merged in a pull request is a topic branch, it is common to delete it after the pull request is merged because you can assume that the work on that branch has been completed. For new pieces of work, you will make new branches and go through the pull request process again. This keeps the remote repository organized and uncluttered by old branches.

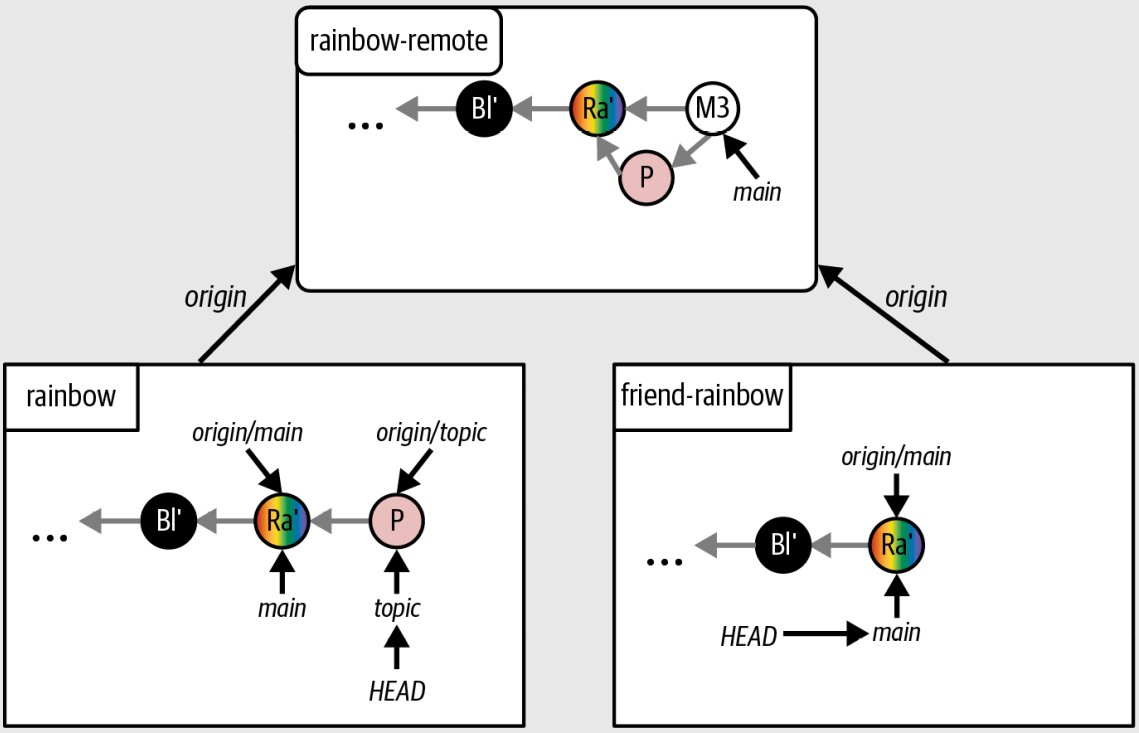
When you created your pull request, if your hosting service offered you the option of automatically deleting the branch once the pull request was merged, then you may no longer have a remote topic branch. If this is not the case, or you chose not to select that option, go on to Follow Along 12-6 to delete the remote topic branch.

[FOLLOW ALONG 12-6]

1. Go to your remote repository on your hosting service and delete the remote topic branch. If necessary, consult the hosting service's documentation for details on how to do this.

Visualize it 12-6 illustrates the state of the repositories after the deletion of the remote topic branch.

[VISUALIZE IT 12-6]



The Rainbow project after you delete the remote topic branch

As you can see, the local main branches in both of the local repositories are now out of sync with the main branch in the remote repository. Also, the local topic branch and origin/topic remote-tracking branch are still in the rainbow repository. Next, you'll make sure everyone pulls the latest changes from the remote repository so that the primary line of development in the Rainbow project, the main branch, is up to date in both local repositories. You'll also clean up the rainbow repository by deleting the local topic branch and the origin/topic remote-tracking branch.

Syncing the Local Repositories and Cleaning Up

The final step of the pull request process is for you and your friend to sync your local repositories with the remote repository. First you will sync the rainbow repository, and since the topic branch has been merged you will also delete the local topic branch and the origin/topic remote-tracking branch.

Recall that in Chapter 8, you used the git fetch command with the -p (which stands for "prune") option to delete any remote-tracking branches that correspond to remote

branches that have been deleted in the remote repository. This time around you will use the -p option with the git pull command, and it will have the same effect. Go to Follow Along 12-7 to sync the rainbow repository.

[FOLLOW ALONG 12-7]

1. rainbow $ **git switch main**
   Switched to branch 'main'
   Your branch is up to date with 'origin/main'.

2. rainbow $ **git pull -p**
   From github.com:gitlearningjourney/rainbow-remote
   - [deleted] (none) -> origin/topic
   remote: Enumerating objects: 1, done.
   remote: Counting objects: 100% (1/1), done.
   remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
   Unpacking objects: 100% (1/1), 626 bytes | 626.00 KiB/s, done.
   7c09136..2f833d6 main -> origin/main
   Updating 7c09136..2f833d6
   Fast-forward
   othercolors.txt | 2 +-
   1 file changed, 1 insertion(+), 1 deletion(-)

3. rainbow $ **git branch -d topic**
   Deleted branch topic (was 4c35a5c).

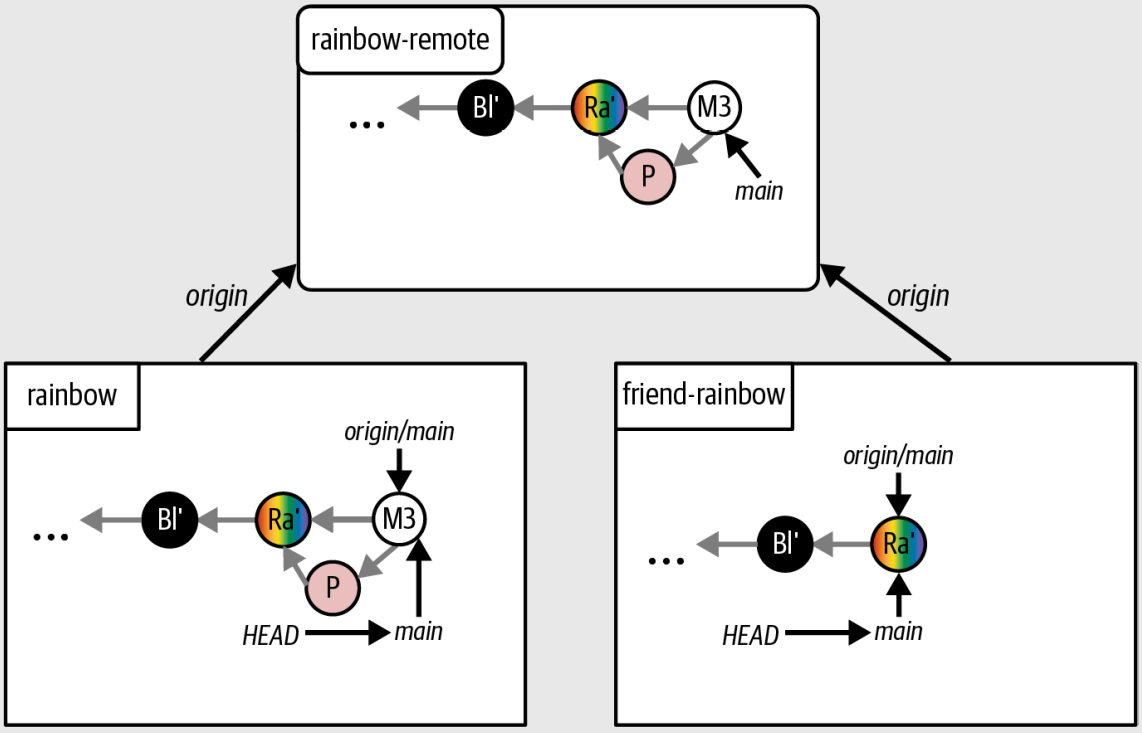
4. rainbow $ **git log**
   commit 2f833d6fa783882c5f832da9e1eafe6d405d3468 (**HEAD -> main**, **origin/main**)
   Merge: 7c09136 4c35a5c
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Mon Jul 4 05:50:08 2022 +0200
   Merge pull request #1 from gitlearningjourney/topic
   Adding the color pink
   commit 4c35a5c02c3dc03f044cbdfdbb0ae55161af6a86
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sun Jul 3 14:16:01 2022 +0200
   pink

What to notice:

- You deleted the local topic branch and the origin/topic remote-tracking branch.
- You updated your local main branch.
- The merge commit (M3) has a commit message and a description that were automatically generated by the hosting service. In the example in this book the commit message is Merge pull request #1 from gitlearningjourney/topic and the commit description is Adding the color pink, which was the title of the pull request you created. Each hosting service may have a slightly different template on which they base this default commit message.
- The parents of the M3 merge commit are 7c09136 (the rainbow commit) and 4c35a5c (the pink commit). As usual, the commit hashes in your repository will be different from the ones in this book because commit hashes are unique.

The state of the Rainbow project after Follow Along 12-7 is illustrated in Visualize it 12-7.

[VISUALIZE IT 12-7]



The Rainbow project after you pull the changes from the remote main branch to the local main branch in the rainbow repository and delete the local topic branch and the origin/topic remote-tracking branch

What to notice:

- In the rainbow repository, the main branch points to the latest M3 merge commit.

Next, in Follow Along 12-8, your friend will sync their local main branch with the changes from the remote repository.

[FOLLOW ALONG 12-8]

1. friend-rainbow $ **git pull**

```
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 4 (delta 1), reused 2 (delta 0), pack-reused 0
Unpacking objects: 100% (4/4), 831 bytes | 166.00 KiB/s, done.
From github.com:gitlearningjourney/rainbow-remote
7c09136..2f833d6 main -> origin/main
Updating 7c09136..2f833d6
Fast-forward
othercolors.txt | 3 ++-
1 file changed, 2 insertions(+), 1 deletion(-)
```

2. friend-rainbow $ **git log**

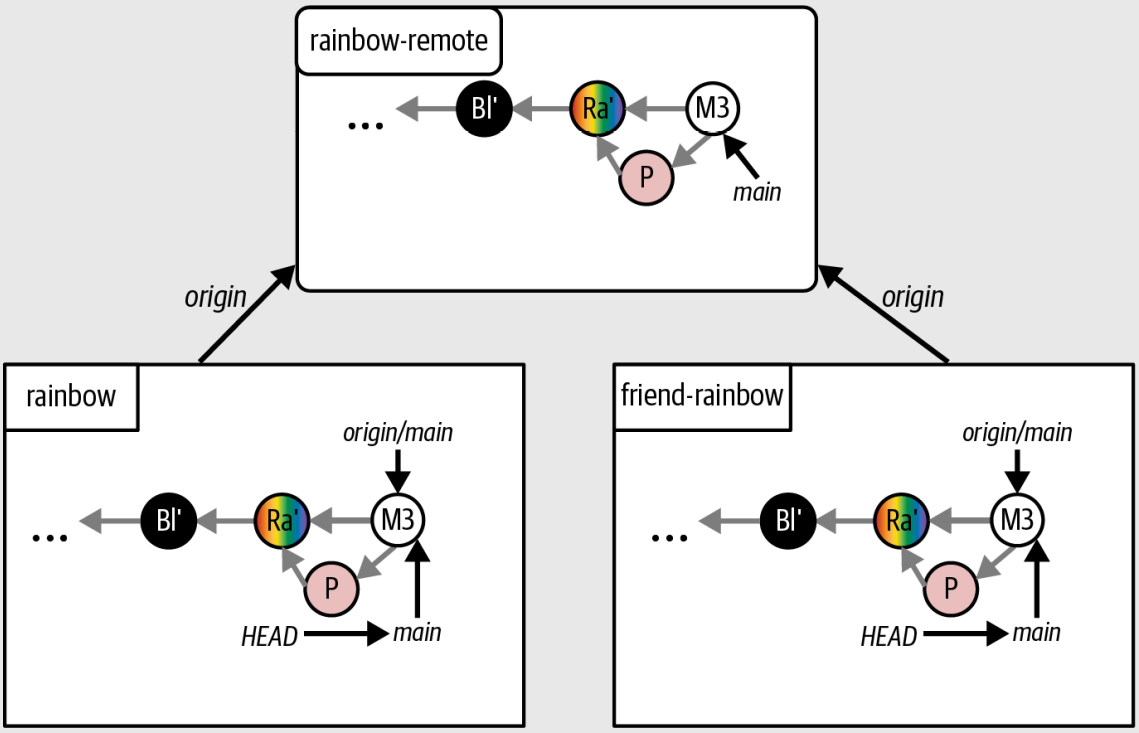
```
commit 2f833d6fa783882c5f832da9e1eafe6d405d3468 (HEAD -> main, origin/main, origin/HEAD)
Merge: 7c09136 4c35a5c
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Mon Jul 4 05:50:08 2022 +0200
Merge pull request #1 from gitlearningjourney/topic
Adding the color pink
commit 4c35a5c02c3dc03f044cbdfdbb0ae55161af6a86
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sun Jul 3 14:16:01 2022 +0200
pink
```

What to notice:

- Your friend updated their local main branch.

This is illustrated in Visualize it 12-8.

[VISUALIZE IT 12-8]

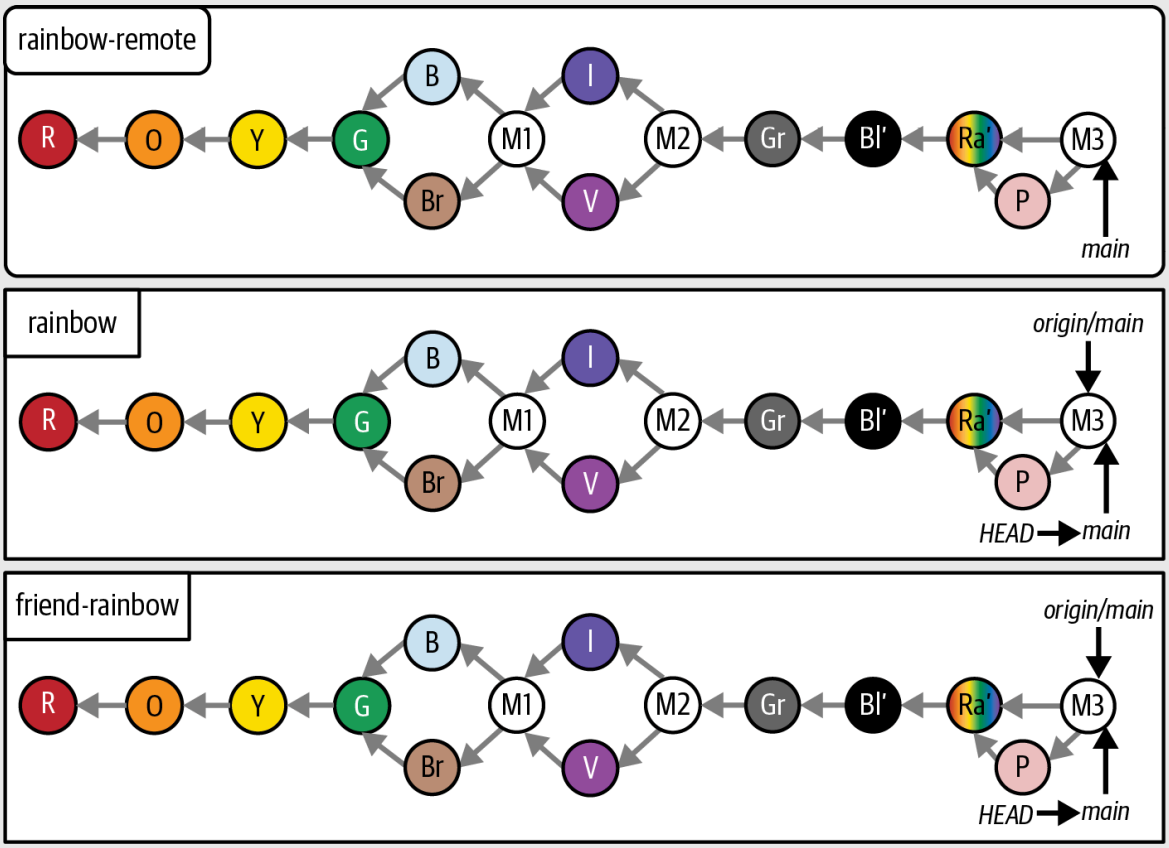


The Rainbow project after your friend pulls the changes from the remote main branch to the local main branch in the friend-rainbow repository

State of the Local and Remote Repositories

You have come to the end of the *Learning Git* experience. Visualize it 12-9 shows the state of the local and remote repositories in the Rainbow project with all the commits that were made from Chapter 1 all the way to the end of Chapter 12.

[VISUALIZE IT 12-9]



The Rainbow project at the end of Chapter 12 with all the commits since Chapter 1

Summary

In this chapter, you learned about pull requests. We discussed why they're a helpful tool for collaborating on Git projects with multiple people, because they facilitate communication and the review process. You carried out all nine steps of the pull request process in a hands-on example in the Rainbow project. While working through this example, you witnessed how by default pull requests are merged with the non-fast-forward option, which produces a merge commit even in the case of a fast-forward merge when the branches involved in the merge have not diverged. You also learned about a simple trick to get Git to generate a command you can use to push a branch to a remote repository and define an upstream branch for it in one go.

Check out the Epilogue for some closing thoughts, and be sure to take a look at the appendixes for some useful reference material.

Previous chapter
< Chapter 11. Rebasing

Next chapter
[Epilogue] >